

# A Balanced Data Diet: Addressing the Exploration Bottleneck in Mega-Scale RL for Robot Control

Octi Zhang<sup>1,2,\*</sup> Mateo Guaman Castro<sup>1,\*</sup> Patrick Yin<sup>1,\*</sup>  
 Ignacio Dagnino<sup>1</sup> Abhishek Gupta<sup>1</sup> Rosario Scalise<sup>1,\*,†</sup> Byron Boots<sup>1,†</sup>  
<sup>1</sup>University of Washington <sup>2</sup>NVIDIA <sup>\*</sup>Equal contribution <sup>†</sup>Equal advising

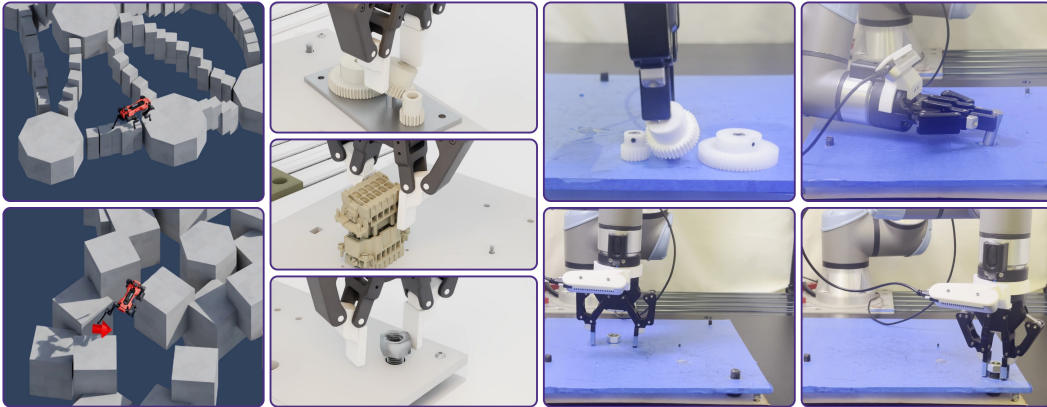


Figure 1: *SGS* enables large-scale RL to learn challenging robotic behaviors, from traversing custom-designed difficult terrains to dexterous, contact-rich assembly. By focusing training on configurations at the frontier of the policy’s capabilities, *SGS* solves tasks that prior sampling methods struggle to learn. We further distill the manipulation policies into RGB-based policies and transfer them zero-shot to real hardware.

**Abstract:** General-purpose robots must perform a wide range of tasks from agile locomotion to dexterous manipulation. While sim-to-real reinforcement learning (RL) has proven to be a useful tool for this goal, current RL pipelines depend on engineering-heavy, per-task structural priors such as shaped rewards and demonstrations. Recent work has shown that diverse simulator resets, combined with massively parallel simulation, can alleviate much of this engineering burden on several manipulation problems. However, we find that naively scaling this paradigm to more precise or dynamic problems remains non-trivial. While simulator resets can help with exploration, uniformly sampling over this distribution wastes a growing fraction of learning experience on task configurations the policy has already mastered or cannot yet attempt. This makes it challenging to see the expected benefits of scaling parallel environments for RL, since much of the learning signal in a batch is wasted during learning. To mitigate this, we introduce Success Guided Sampling (*SGS*), a simple adaptive sampler that concentrates RL training on task configurations around the frontier of the policy’s capabilities. Doing so allows large-scale simulated RL to make the most out of the experience in a batch, enabling much more effective scaling to large-scale parallel simulation. Across experiments using up to  $2^{20}$  (over one million) parallel environments, *SGS* enables RL to solve challenging multi-terrain quadruped locomotion and contact-rich assembly tasks that prior methods fail to solve. Finally, we distill the learned manipulation policies into RGB-based policies and demonstrate zero-shot transfer to several challenging assembly tasks on real hardware. Project website: <https://sgs-rl.github.io/>.

**Keywords:** Reinforcement Learning, Physics-Based Simulation, Manipulation, Locomotion

# 1 Introduction

General-purpose robotics requires a training recipe that works across diverse embodiments and tasks, from legged systems navigating complex terrain to dexterous arms manipulating contact-rich objects. Sim-to-real reinforcement learning with massively parallel physics simulation [1, 2, 3, 4] has emerged as a promising pathway toward this goal, producing policies that transfer to real hardware across agile locomotion [5, 6, 7], dexterous manipulation [8, 9, 10], and contact-rich assembly [11, 12].

However behind these successes of RL lies substantial per-task engineering. Reward functions require expert tuning and frequently induce unintended behaviors [13, 14]. Curricula, whether hand-designed [15] or automatic within a fixed parameterization [16, 17, 8], are tuned or specified per task. Demonstrations, when used to bootstrap exploration, both constrain the resulting behavior and require collection effort [18, 19, 20, 21]. And achieving breadth across tasks typically requires training single-task experts and distilling into a generalist [14], bloating the training pipeline. Moreover, each of these choices induces a new set of hyperparameters that requires tuning. Each new task is a fresh engineering project, making this paradigm far from a turnkey solution.

A growing body of work uses simulator resets to make difficult robotic tasks learnable, exposing policies to useful intermediate states rather than requiring successful exploration from scratch [15, 20, 22, 12]. These approaches construct starting states through demonstrations, geometric constraints, or programmatic sampling. OmniReset [12] shows that diverse, programmatically generated resets combined with massively parallel RL can substantially reduce task-specific reward and demonstration engineering. Together, these results establish reset distributions as an important component of scaling robot learning.

Providing useful reset states is only part of the problem: training must also allocate experience across them. Under uniform sampling, many environments may start from configurations the policy has already mastered or cannot yet solve, leaving fewer opportunities to practice behaviors it is still learning. In our experiments, this limits the gains from increasing the number of parallel environments, particularly on challenging manipulation and locomotion tasks.

We introduce *SGS*, a simple sampler that adapts task-configuration sampling throughout training using non-parametric estimates of the policy’s current success rates. Intuitively, learning signal is concentrated in regions of the space that are neither too hard nor too easy under the current policy. We instantiate this by weighting success estimates with a beta distribution, which prioritizes configurations with moderate success rates. This ensures that sampling is generically guided by the policy’s performance rather than a manually ordered task curriculum. We study the same instantiation of *SGS* across difficult-terrain quadruped whole-body locomotion and dexterous NIST assembly tasks [23], including threaded assembly, insertion, connector assembly, and gear meshing. We train without demonstrations, using one simple reward function shared across all locomotion tasks and another shared across all manipulation tasks, at scales of up to approximately one million parallel environments. *SGS* enables improved learning throughput, solving much more challenging tasks at scale than possible before.

We outline our contributions as follows:

1. We introduce *SGS*, a simple success-based sampler for large-scale robot RL, and study how adaptive sampling of task configurations scales to approximately one million parallel environments.
2. We evaluate *SGS* across difficult-terrain quadruped locomotion and diverse assembly tasks (Figure 3). *SGS* outperforms uniform sampling and Prioritized Level Replay (PLR) [24] on challenging multi-terrain whole-body locomotion and nut-and-bolt assembly, while performing comparably to uniform sampling on simpler tasks.
3. We distill learned manipulation policies into RGB-based policies and demonstrate zero-shot transfer to several precise assembly tasks on real robot hardware (Figure 4).

## 2 Related Work

**Sim-to-real reinforcement learning.** Sim-to-real RL has enabled substantial advances in legged locomotion [5, 25, 26, 14, 27, 18, 28, 29] and dexterous manipulation [8, 10, 9, 11, 12], often using PPO [30]. These successes commonly rely on task-specific reward shaping, curricula, or demonstrations [14, 31, 9, 10, 11, 32]. We study a training recipe that combines diverse simulator resets, success-based adaptive sampling, and large-scale RL, using no demonstrations for RL training and one shared reward function within each domain.

**Reset distributions for exploration.** The choice of training-state distribution has long been recognized as important for policy learning [33]. Resetting to useful intermediate states can make difficult exploration problems tractable, as demonstrated by backward learning from a single demonstration and Go-Explore [34, 35]. In robotics, reverse curricula, RFCL, and DemoStart use goal states or demonstration states to construct starting-state curricula [15, 20, 21], while sampling-based exploration and constrained sampling construct useful reset states from the geometry of manipulation [36, 22]. OmniReset generates diverse resets programmatically and combines them with massively parallel RL [12]. Building on this body of work, *SGS* maintains empirical success estimates over a fixed set of task configurations and continuously adjusts how frequently each configuration is sampled during training.

**Scaling on-policy RL with parallel simulation.** Massively parallel simulation has made large-scale on-policy RL practical for robot learning [5, 37]. Recent work improves learning at scale through parallel exploration and population-based training [38, 39], changes to loss estimation [40], and staggered resets that increase temporal diversity within training batches [41]. *SGS* studies a complementary component of this training process: allocating parallel environments across task configurations according to the policy’s current success rates.

**Automatic curricula and task sampling.** Automatic curriculum learning selects training tasks according to the learner’s capabilities [42]. Approaches include learning-progress-based task selection [16, 43], asymmetric self-play that generates increasingly challenging goals [44], and unsupervised environment design. Within the latter, PAIRED generates environments using a regret objective, PLR prioritizes previously encountered levels based on TD-errors, and ACCEL evolves levels to construct a curriculum [45, 24, 46]. Most closely related to *SGS*, Sampling for Learnability (SFL) prioritizes levels using  $p(1 - p)$ , where  $p$  is empirical success rate [47]. *SGS* follows the same principle of emphasizing moderate success rates, using rolling success estimates and a smooth beta-shaped weighting over a fixed configuration set. Its target success rate is adjustable, and every configuration retains nonzero sampling probability. We evaluate this simple approach against uniform sampling and PLR on our scaling benchmarks, with an additional SFL comparison on locomotion.

We do not claim a fundamentally new curriculum principle. Instead, we show that a simple variant of success-based sampling, combined with diverse resets and large-scale RL, can solve challenging locomotion and assembly tasks that prior sampling methods struggle to learn.

## 3 Method

### 3.1 Problem Setup

**Reinforcement Learning Problem:** We study the problem of learning goal-conditioned RL policies from scratch for challenging robotic control tasks. We instantiate this problem as a goal-conditioned Markov Decision Process (MDP)  $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{G}, P, r, \gamma, \rho)$  where  $s \in \mathcal{S}$  is the state,  $a \in \mathcal{A}$  is the robot’s action,  $g \in \mathcal{G}$  is the robot’s goal,  $P$  is the transition dynamics,  $r$  is the reward function,  $\gamma$  is the discount factor, and  $\rho$  is the distribution over initial state and goal pairs  $(s_0, g) \in \mathcal{S} \times \mathcal{G}$ .  $s_0$  captures all episode-level environment variation, including terrain geometry for locomotion and object instances for manipulation. The reward  $r(s_t, a_t, g)$  is shared within each domain: one reward covers all locomotion tasks and another covers all manipulation tasks (Equation 3). The objective is

to maximize the expected discounted sum of rewards  $J(\pi) = \mathbb{E}_{(s_0, g) \sim \rho, a \sim \pi} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, g)]$ . We focus our RL training on compact Lagrangian states.

**Task configurations.** We represent each task configuration as  $\tau = (s_0, g, e)$ , where  $s_0$  is the initial state,  $g$  is the goal, and  $e$  specifies the environment configuration. Each episode starts from one such configuration, and *SGS* adapts how frequently configurations are sampled as the policy learns.

For locomotion, a task configuration specifies the terrain type and geometry, the robot’s starting pose, and its goal pose. Sampling configurations therefore determines both which terrain the robot practices on and where it must travel within that terrain.

For manipulation, a task configuration specifies the assembly task, the initial robot and object states, and the receptacle pose. The goal is the assembled object pose relative to the receptacle, whose pose is represented in the policy’s observations. Initial states cover Reaching, Stable Grasp, and Near-Goal resets, following OmniReset [12]. Sampling configurations determines which starting conditions the policy practices to complete the assembly.

In both domains, *SGS* allocates training episodes across these configurations using the policy’s current success rates. Appendices A.1 and A.2 provide the task-specific configuration and reset definitions.

### 3.2 SGS: Success Guided Sampling

The core insight of *SGS* is that learning complex control requires exposing the policy to the right experience at the right time. In a goal-conditioned setting, success rate is a cheap, online signal of which configurations the policy should practice now. The aim is to spend most environment steps on configurations that are neither too easy nor too hard, while occasionally revisiting all of them to keep success-rate estimates accurate.

Before training, we sample a fixed set of  $N$  configurations  $\{\tau_i\}_{i=1}^N$  from the task distribution. At each episode reset, *SGS* selects a configuration from this set. For each configuration, we maintain the last  $H$  Boolean outcomes and estimate its success rate  $p_i$  as their mean.

A configuration’s success rate reflects both its difficulty and the policy’s training history. It can change as the policy practices that configuration or learns transferable skills from others. *SGS* therefore updates its estimates throughout training, concentrating experience on configurations with moderate success rates while continuing to revisit easier and harder configurations.

We then score each configuration with a Beta-shaped kernel in mode-concentration form,  $\text{Beta}(1 + \kappa t, 1 + \kappa(1 - t))$ , parameterized by a target success rate (i.e. mode)  $t \in [0, 1]$  and concentration  $\kappa \geq 0$ :

$$w_i = (p_i + \epsilon)^{\kappa t} (1 - p_i + \epsilon)^{\kappa(1-t)}, \quad \ell_i = \log(w_i + \epsilon), \quad (1)$$

where  $w_i$  is the unnormalized density at  $p_i$  and  $\ell_i$  the corresponding logit. The small  $\epsilon > 0$  both prevents singularities at  $p_i \in \{0, 1\}$  and keeps every configuration’s sampling probability non-zero, so that success-rate estimates can be updated even for configurations the robot cannot yet solve. At the end of each episode, the next configuration is sampled i.i.d. from a softmax over  $\{\ell_i\}$  with temperature  $T$ :

$$p(i) = \frac{\exp(\ell_i/T)}{\sum_{j=1}^N \exp(\ell_j/T)} \quad (2)$$

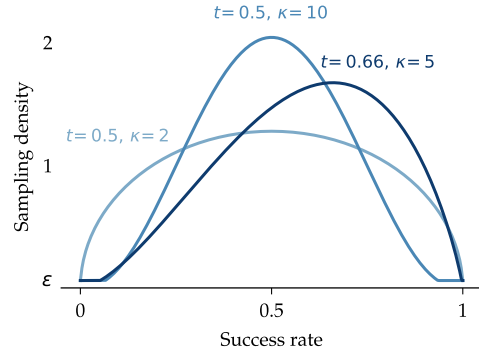


Figure 2: **Beta-shaped sampling distribution.** Different parameterizations prioritize coverage and sampling greediness.

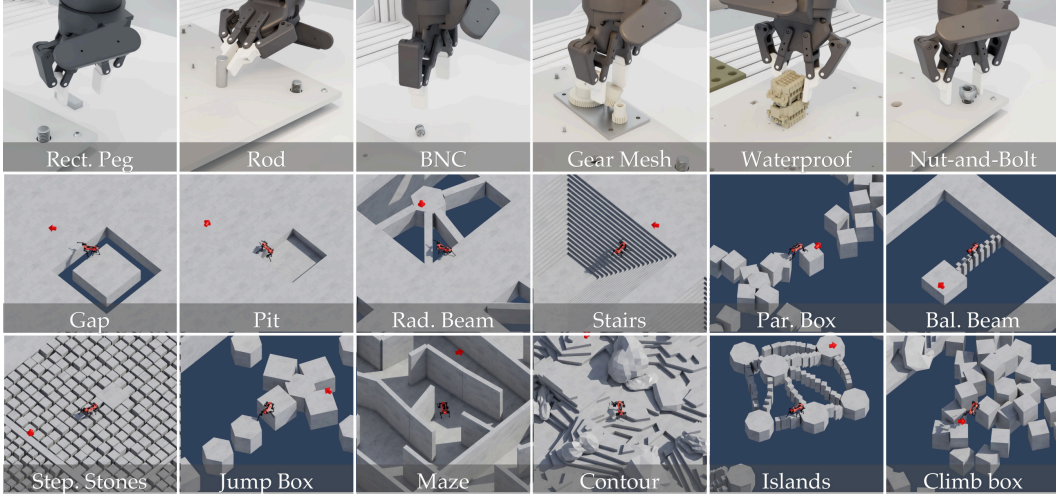


Figure 3: **Manipulation and locomotion tasks.** 6 NIST Assembly Task-Board (ATB1) [23] tasks and 12 of the 13 challenging locomotion terrains, solved with *SGS*. Manipulation tasks require precision and dexterity to solve tasks with tight tolerances, such as screwing a nut onto a bolt. Locomotion tasks require the robot to string together multiple agile maneuvers as well as precise foothold placements to traverse complex terrains such as uneven floating islands.

Intuitively, configurations with very high or very low success rates are sampled with small but non-zero probability, while those near the target success rate are sampled most often (Figure 2). In practice, we use  $N = 32,768$  for manipulation and  $N = 104,000$  for locomotion,  $H = 100$ , and  $T = 2$ . We use  $t = 0.66$ ,  $\epsilon = 10^{-8}$ , and  $\kappa = 5$  for locomotion, and  $t = 0.5$ ,  $\epsilon = 10^{-4}$ , and  $\kappa = 1$  for manipulation. We find *SGS* to not be sensitive to most of these hyperparameters. Appendix B.2 reports sensitivity to these choices and implementation details.

### 3.3 Algorithmic Decisions for RL Training

Beyond the sampler itself, *SGS* makes two deliberate algorithmic choices that work together to support training across a wide task configuration space with PPO [30].

**Shared rewards within each domain.** We use one simple reward function across all locomotion tasks and another across all manipulation tasks. Each combines a boolean task success reward obtained at the end of the episode with relatively small robot-specific, per-step regularization terms:

$$r(s_t, a_t, g) = \lambda_{\text{task}} r_{\text{task}}(s_t, a_t, g) + \lambda_{\text{reg}} r_{\text{reg}}(s_t, a_t). \quad (3)$$

The shared-reward design avoids specifying a different reward function for each terrain or assembly task. Appendix A.3 gives the terms and weights for each domain.

**Scaling parallel environments.** When combined with adaptive sampling, we find that scaling the number of parallel environments enables *SGS* to solve substantially harder tasks than would otherwise be possible. Intuitively, when many task configurations produce no useful gradient signal (because the policy has already mastered them or cannot yet solve them), uniform sampling wastes a growing fraction of environments on these configurations as scale grows. *SGS* addresses this by concentrating rollouts on configurations where the policy is actively learning, so additional environments translate into denser learning signal rather than diminishing returns. This complements recent work on optimizer-side fixes to PPO saturation, including approaches addressing data redundancy from a single Gaussian policy [38] and, in recent work, degraded loss estimates at scale [40].

## 4 Experimental Results

Through a series of large-scale scaling studies, we ask: **(Q1) Scaling** — does *SGS* scale better than uniform sampling and PLR [24] as parallel training environments grow? **(Q2) Capability** —

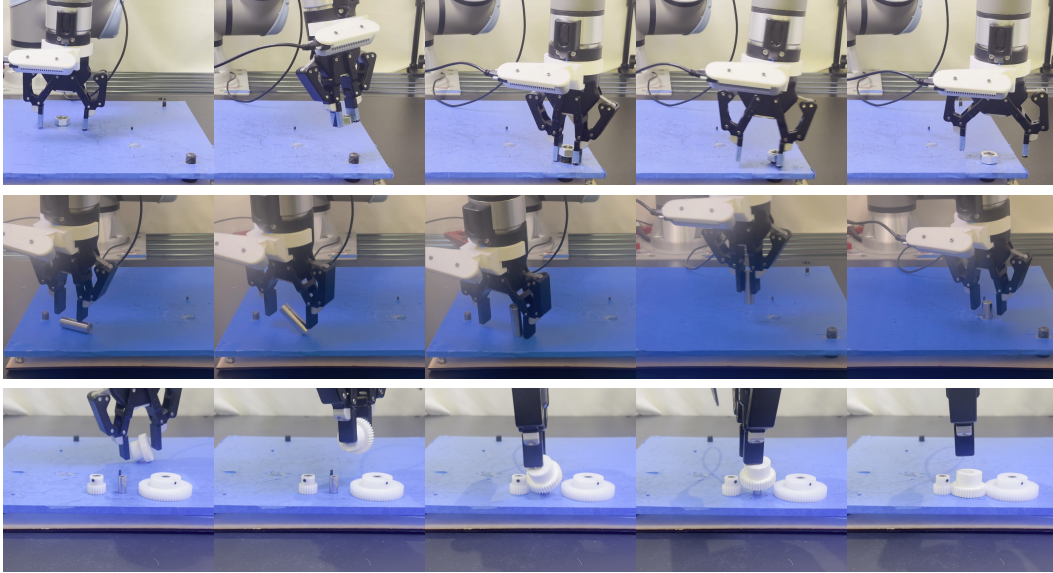


Figure 4: **Zero-shot RGB sim-to-real transfer.** Successful nut-and-bolt assembly (top), rod-in-hole insertion (middle), and gear mesh insertion (bottom) trajectories on UR5e hardware.

does *SGS* unlock robotics tasks beyond what prior methods could solve, even at smaller scales? **(Q3) Transfer** — can dexterous manipulation behaviors learned with *SGS* transfer zero-shot to real hardware?

#### 4.1 Locomotion

We study locomotion for a quadrupedal robot traversing a variety of different terrains, some of which can be seen in Figure 1 (See Appendix A.1 for a thorough description). We use a set of 13 terrain types, which span a wide range of difficulties and capabilities that the robot needs to obtain in order to traverse them successfully. These include terrains found in previous works, such as slopes, pits, gaps, stairs, [48], radiating beams, balance beams, stepping stones [27], and five harder procedurally generated terrains. These include navigation mazes, contoured terrains, and four variations of floating islands connected to each other via either narrow stepping stones or randomly arranged jump boxes [26, 7, 49] at different heights. These terrains test both precise locomotion and navigation, requiring the robot to string together multiple agile maneuvers. Notably, we train a *single* policy for all of these terrains from scratch, unlike many prior methods which train per-terrain specialist policies and then distill them into a single policy [48, 27, 26, 49]. We use the AnyMal-D robot [50] with system-identified actuators [51], to ensure realistic motions.

#### 4.2 Manipulation

We study contact-rich assembly on six UR5e tasks (Figure 3, top row) and Franka nut-and-bolt assembly (Figure 6) from the NIST Assembly Taskboard [23]. The UR5e tasks cover threading, precise insertion, connector mating, and gear alignment. The UR5e uses a Robotiq 2F-85 gripper and operational-space control; the Franka uses a parallel-jaw gripper and joint-position control.

Each task requires grasping and reorienting a part, transporting it to its mating geometry, and completing a precise, contact-rich assembly. In nut-and-bolt assembly, the robot threads an M16 nut onto an upright bolt through approximately 6.5 revolutions. In gear mesh insertion, it seats a medium gear on its shaft with

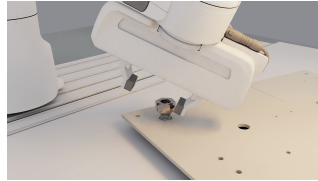


Figure 6: nut-and-bolt assembly with Franka evaluated in our scaling experiment.

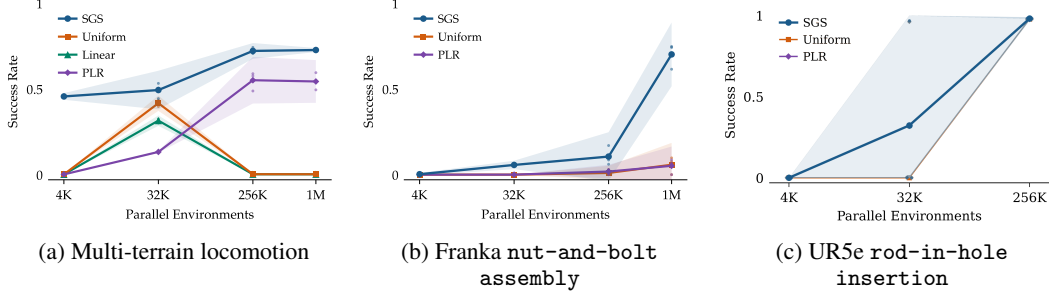


Figure 5: Success rate versus the number of parallel training environments. Curves show means over three training seeds, dots show individual seeds, and shaded regions show 95% confidence intervals. *SGS* achieves higher mean success than the compared baselines at 1M environments on locomotion and Franka nut-and-bolt assembly. On the simpler UR5e rod-in-hole insertion, *SGS* outperforms uniform sampling and PLR at smaller scale (32K), and all methods reach approximately 98% success at high-enough scale (256K).

its teeth meshing with the adjacent small and large gears. In `waterproof insertion`, it inserts the connector into its socket until fully seated. In `BNC assembly`, it inserts the connector and rotates it to engage the lock. In `rod-in-hole insertion`, it inserts a 16 mm rod into the taskboard hole. In `rectangular-peg-in-hole insertion`, it inserts a rectangular peg into its opening, requiring positional and rotational alignment. Appendices A.2 and A.3.2 give task-specific reset and success definitions.

We use the same simple reward function and diverse simulator resets across all manipulation tasks. Our main scaling manipulation experiments use a gravity curriculum which we later found to be unnecessary; Appendix B.4 describes its implementation and ablates its effect.

### 4.3 *SGS* benefits from increased training scale.

We compare the scaling behavior of *SGS* against three baselines: uniform sampling, hand-designed curricula (common in locomotion RL), and Prioritized Level Replay (PLR) [24], a popular Unsupervised Environment Design method [45]. Uniform sampling draws task configurations from  $\rho$  at every reset. It has recently been shown to suffice for easier manipulation tasks [12] but has not been effective in multi-task locomotion. Locomotion instead commonly uses a “linear” curriculum [48, 5], in which terrain difficulty is interpolated along a hand-designed heuristic such as stair height. Robots start at random levels and are promoted when they get within some distance of the goal or demoted when they make little progress. PLR samples the next task configuration using a “learnability score” plus a staleness bonus; we use the absolute GAE as the score.

We measure average success on Anymal-D locomotion and Franka Panda nut-and-bolt assembly across four training scales, up to 1M parallel environments. We report means over three training seeds per scale for both tasks. Evaluation resets are freshly sampled from the training distribution rather than reused from training. We hold the number of PPO mini-batches constant, so batch size scales with environment count, and fix all other PPO and optimizer hyperparameters. We train locomotion for 5K PPO iterations and nut-and-bolt assembly for 7.5K, with up to 32,768 and 16,384 environments per GPU, respectively, on L40S and H200 GPUs. Training takes about 36 hours for locomotion and 128 hours for nut-and-bolt assembly.

At 1M parallel environments, *SGS* reaches 73% mean success on multi-task locomotion versus 54% for PLR (Figure 5). On Franka Panda nut-and-bolt assembly, it reaches 70%, versus 6% for uniform sampling and 5% for PLR. Notably, we see monotonic improvement in performance with increased scale, where other methods struggle to benefit or collapse. On the easier UR5e rod-in-hole insertion task (Figure 5(c)), *SGS* succeeds at 32K environments, where uniform sampling and PLR achieve zero success, while all methods reach about 98% at 256K. *SGS* also outperforms Sampling for Learnability (SFL) [47] on locomotion at 4K and 32K environments (Table 1). We report both SFL’s best and final performance to show its late-training decline; implementation details are in Appendix B.1.

Table 1: Locomotion success rate (mean  $\pm$  95% confidence interval over three training seeds). SFL best is reported alongside final performance to show the effect of its late-training decline.

| Parallel environments | SFL final         | SFL best          | SGS final         |
|-----------------------|-------------------|-------------------|-------------------|
| 4K                    | $0.255 \pm 0.051$ | $0.289 \pm 0.015$ | $0.456 \pm 0.019$ |
| 32K                   | $0.051 \pm 0.142$ | $0.421 \pm 0.032$ | $0.494 \pm 0.113$ |

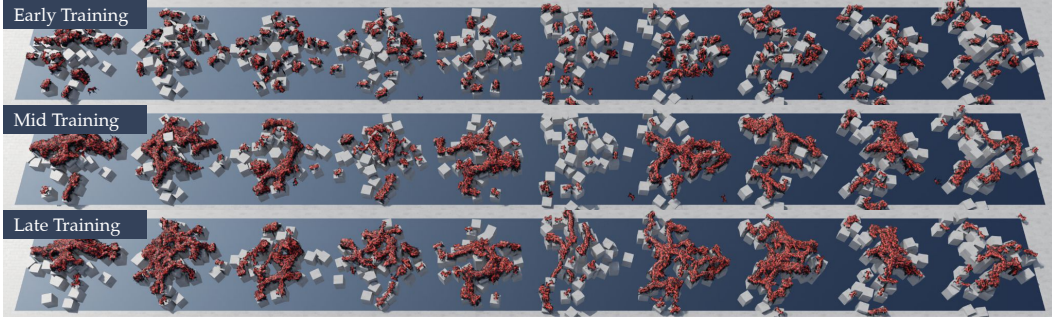


Figure 7: **Sampling of configurations under SGS over training.** Early in training, *SGS* samples uniformly over all task configurations. Halfway through training, *SGS* focuses sampling mass on learning to jump across islands that are level to one another. In late training, *SGS* samples configurations that include the top of the center pillar, which is at a significant height difference compared to the other islands.

#### 4.4 *SGS* unlocks new capabilities even at small scale.

*SGS*'s advantage over baselines emerges well before the saturation effects seen at large scale, as the UR5e rod-in-hole insertion scaling comparison shows (Figure 5(c)). The same holds for multi-task locomotion: with only 4,096 parallel environments, *SGS* achieves non-trivial success on many terrains while uniform sampling and other baselines fail to learn at all (Figure 8). Even at small scale, *SGS*'s adaptive sampling squeezes more useful learning signal under the same budget compared to uniform sampling.

#### 4.5 Zero-shot deployment on real hardware

For real-world transfer, we train additional UR5e assembly policies with *SGS*. Appendix B.5 reports their RL training setup and simulation comparisons. We distill the state-based teachers into RGB policies with DAgger and transfer them zero-shot to UR5e hardware for nut-and-bolt assembly, rod-in-hole insertion, and gear mesh insertion. Distillation takes about two days on four L40S GPUs with 480 environments each. Table 2 reports teacher and RGB-student success in simulation alongside real-world success, first-try success (task completed on the first insertion attempt), and throughput (successes per minute of total evaluation time, including resets). *SGS* policies exhibit robust retrying and dexterous non-prehensile manipulation, such as flipping the nut and pushing the gear against the shaft to reorient it (Figure 4). See Appendix C for distillation, sim-to-real transfer, hardware, and evaluation details.

Table 2: **Simulation and zero-shot hardware performance.** Simulation success is reported from the Reaching initial-state distribution. Hardware success counts and trial totals are shown in parentheses.

| Task                  | Simulation success (%) |             | Real hardware |                       |                            |
|-----------------------|------------------------|-------------|---------------|-----------------------|----------------------------|
|                       | Teacher                | RGB student | Success (%)   | First-try success (%) | Throughput (successes/min) |
| nut-and-bolt assembly | 90.04                  | 89.05       | 37.5 (18/48)  | 12.5                  | 0.23                       |
| rod-in-hole insertion | 95.70                  | 93.36       | 61 (30/49)    | 28.57                 | 0.71                       |
| gear mesh insertion   | 96.09                  | 96.42       | 94 (47/50)    | 76.0                  | 3.08                       |

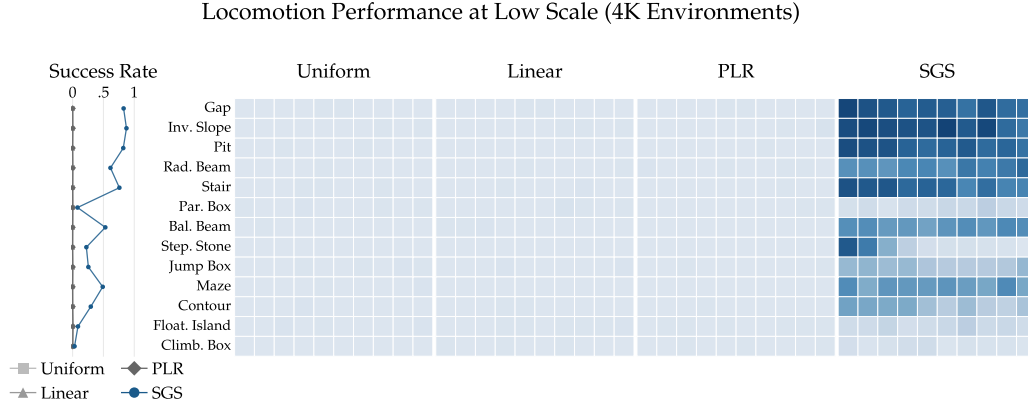


Figure 8: **Locomotion training at 4K environments.** Mean success rate at 4096 parallel environments. *SGS* achieves non-zero success on a large portion of the terrains, while baselines do not learn at all at this scale.

## 5 Limitations

We find three primary limitations. First, *SGS* maintains success-rate estimates over a discrete set of  $K$  cells; while  $K = 32,768$  is sufficient here, continuous or much larger task spaces would require a learned or hierarchical scoring representation. Second, *SGS* samples from a given distribution but does not construct it: we rely on reset-diversity methods such as OmniReset [12] for broad coverage, and closing the loop on *which* configurations to make available remains an open problem. Third, a sim-to-real transfer gap remains, and closing it to match simulated success rates is left to future work.

## 6 Conclusion

We present *SGS*, a simple success-based sampler that focuses training on task configurations with moderate success rates. Combined with diverse resets and large-scale RL, *SGS* solves harder tasks than prior samplers under the same training budget, including quadruped locomotion and nut-and-bolt assembly. We distill the manipulation policies to RGB and demonstrate zero-shot transfer of three NIST board assembly tasks to UR5e hardware. Training policies with *SGS* enables learning without as much hand-engineered reward specification nor manual curriculum engineering on challenging tasks compared to previous works. These results show that adaptively choosing what the policy practices is an important component of scaling robot RL, and suggest a path toward broader multi-task learning and general-purpose dexterous policies.

## Acknowledgments

We thank the University of Washington’s Hyak and Tillicum GPU-accelerated computing platforms, NVIDIA, and NSF ACCESS for the GPU compute that made it possible to run our experiments. This work used the DeltaAI system at the National Center for Supercomputing Applications through allocation CIS260956 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296 [52]. DeltaAI is supported by the National Science Foundation (award OAC 2320345) and the State of Illinois. Patrick Yin is supported by the National Science Foundation Graduate Research Fellowship Program. Abhishek Gupta acknowledges support from Toyota Research Institute under the University 3.0 Research Program. We thank Nam Pho for administering the ACCESS allocation, Andrey Kolobov at Microsoft Research for lending us a NIST task board, and Emma Romig for help building a prototype NIST board at UW. We thank members of the University of Washington’s Robot Learning and WEIRD Lab for useful discussions and feedback.

## References

- [1] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [2] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg. Orbit: A unified simulation framework for interactive robot learning environments. *arXiv preprint arXiv:2301.04195*, 2023.
- [3] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.
- [4] J. Gu, F. Xiang, X. Li, Z. Ling, X. Liu, T. Mu, Y. Tang, S. Tao, X. Wei, Y. Yao, X. Yuan, P. Xie, Z. Huang, R. Chen, and H. Su. Maniskill2: A unified benchmark for generalizable manipulation skills. In *International Conference on Learning Representations (ICLR)*, 2023.
- [5] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2021.
- [6] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.
- [7] X. Cheng, K. Shi, A. Agarwal, and D. Pathak. Extreme parkour with legged robots. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [8] A. Handa, A. Allshire, V. Makoviychuk, A. Petrenko, R. Singh, J. Liu, D. Makoviichuk, K. Van Wyk, A. Zhurkevich, B. Sundaralingam, Y. Narang, J.-F. Lafleche, D. Fox, and G. State. Dextreme: Transfer of agile in-hand manipulation from simulation to reality. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [9] R. Singh, A. Allshire, A. Handa, N. Ratliff, and K. Van Wyk. Dextrah-rgb: Visuomotor policies to grasp anything with dexterous hands. *arXiv preprint arXiv:2412.01791*, 2024.
- [10] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang. Solving rubik’s cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019.
- [11] Y. Narang, K. Storey, I. Akinola, M. Macklin, P. Reist, L. Wawrzyniak, Y. Guo, A. Moravanschky, G. State, M. Lu, A. Handa, and D. Fox. Factory: Fast contact for robotic assembly. In *Robotics: Science and Systems (RSS)*, 2022.
- [12] P. Yin, T. Westenbroek, Z. Zhang, J. Tran, I. Dagnino, E. Shilamkar, N. Mbiziwo-Tiapo, S. Bagaria, X. Liu, G. Mullins, et al. Emergent dexterity via diverse resets and large-scale reinforcement learning. *arXiv preprint arXiv:2603.15789*, 2026.
- [13] T. He, Z. Wang, H. Xue, Q. Ben, Z. Luo, W. Xiao, Y. Yuan, X. Da, F. Castañeda, S. Sastry, C. Liu, G. Shi, L. Fan, and Y. Zhu. VIRAL: visual sim-to-real at scale for humanoid locomanipulation. *CoRR*, abs/2511.15200, 2025. doi:10.48550/ARXIV.2511.15200. URL <https://doi.org/10.48550/arXiv.2511.15200>.
- [14] N. Rudin, J. He, J. Aurand, and M. Hutter. Parkour in the wild: Learning a general and extensible agile locomotion policy using multi-expert distillation and rl fine-tuning. *arXiv preprint arXiv:2505.11164*, 2025.

- [15] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel. Reverse curriculum generation for reinforcement learning. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 78 of *Proceedings of Machine Learning Research*, pages 482–495. PMLR, 2017.
- [16] A. Graves, M. G. Bellemare, J. Menick, R. Munos, and K. Kavukcuoglu. Automated curriculum learning for neural networks. In *international conference on machine learning*, pages 1311–1320. Pmlr, 2017.
- [17] C. Florensa, D. Held, X. Geng, and P. Abbeel. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pages 1515–1528. PMLR, 2018.
- [18] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics*, 37(4), 2018.
- [19] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Robotics: Science and Systems (RSS)*, 2018.
- [20] S. Tao, A. Shukla, T.-k. Chan, and H. Su. Reverse forward curriculum learning for extreme sample and demonstration efficiency in reinforcement learning. *arXiv preprint arXiv:2405.03379*, 2024.
- [21] M. Bauzá, J. E. Chen, V. Dalibard, N. Gileadi, R. Hafner, M. F. Martins, J. Moore, R. Pevcevičute, A. Laurens, D. Rao, M. Zambelli, M. A. Riedmiller, J. Scholz, K. Bousmalis, F. Nori, and N. Heess. Demostart: Demonstration-led auto-curriculum applied to sim-to-real with multi-fingered robots. In *IEEE International Conference on Robotics and Automation, ICRA 2025, Atlanta, GA, USA, May 19-23, 2025*, pages 6756–6763. IEEE, 2025. doi:10.1109/ICRA55743.2025.11127813. URL <https://doi.org/10.1109/ICRA55743.2025.11127813>.
- [22] M. Toussaint, C. V. Braun, A. Jordana, S. Auddy, E. Cobo-Briesewitz, D. Shcherba, T. Burghoff, and J. Carpentier. Combined constrained sampling and reinforcement learning for robotic manipulation. *arXiv preprint arXiv:2602.08557*, 2026.
- [23] K. Kimble, K. Van Wyk, J. Falco, E. Messina, Y. Sun, M. Shibata, W. Uemura, and Y. Yokokohji. Benchmarking protocols for evaluating small parts robotic assembly systems. *IEEE Robotics and Automation Letters*, 5(2):883–889, 2020.
- [24] M. Jiang, E. Grefenstette, and T. Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pages 4940–4950. PMLR, 2021.
- [25] A. Kumar, Z. Fu, D. Pathak, and J. Malik. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.
- [26] D. Hoeller, N. Rudin, D. Sako, and M. Hutter. Anymal parkour: Learning agile navigation for quadrupedal robots. *Science Robotics*, 9(88):eadi7566, 2024.
- [27] C. Zhang, N. Rudin, D. Hoeller, and M. Hutter. Learning agile locomotion on risky terrains. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11864–11871. IEEE, 2024.
- [28] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025.
- [29] Y. Li, Z. Luo, T. Zhang, C. Dai, A. Kanervisto, A. Tirinzoni, H. Weng, K. Kitani, M. Guzek, A. Touati, et al. Bfm-zero: A promptable behavioral foundation model for humanoid control using unsupervised reinforcement learning. *arXiv preprint arXiv:2511.04131*, 2025.

- [30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [31] T. He, Z. Wang, H. Xue, Q. Ben, Z. Luo, W. Xiao, Y. Yuan, X. Da, F. Castañeda, S. Sasstry, et al. Viral: Visual sim-to-real at scale for humanoid loco-manipulation. *arXiv preprint arXiv:2511.15200*, 2025.
- [32] B. Tang, B. Wen, I. Akinola, J. Xu, K. Van Wyk, A. Handa, G. S. Sukhatme, F. Ramos, D. Fox, and Y. Narang. Automate: Specialist and generalist assembly policies over diverse geometries. In *Robotics: Science and Systems*, 2024.
- [33] J. A. Bagnell, S. Kakade, A. Y. Ng, and J. Schneider. Policy search by dynamic programming. In *Advances in Neural Information Processing Systems*, volume 16, 2003.
- [34] T. Salimans and R. Chen. Learning Montezuma’s Revenge from a single demonstration. *arXiv preprint arXiv:1812.03381*, 2018.
- [35] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune. First return, then explore. *Nature*, 590:580–586, 2021.
- [36] G. Khandate, S. Shang, E. T. Chang, T. L. Saidi, Y. Liu, S. M. Dennis, J. Adams, and M. Cio-earlie. Sampling-based exploration for reinforcement learning of dexterous manipulation. In *Robotics: Science and Systems*, 2023.
- [37] S. Tao, F. Xiang, A. Shukla, Y. Qin, et al. ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI. *arXiv preprint arXiv:2410.00425*, 2024.
- [38] J. Singla, A. Agarwal, and D. Pathak. Sapg: Split and aggregate policy gradients. In *International Conference on Machine Learning*, pages 45759–45772. PMLR, 2024.
- [39] A. Petrenko, A. Allshire, G. State, A. Handa, and V. Makoviychuk. Dexpbt: Scaling up dexterous manipulation for hand-arm systems with population based training. *arXiv preprint arXiv:2305.12127*, 2023.
- [40] M. Beukman, K. Khetarpal, Z. Zheng, W. Dabney, J. Foerster, M. Dennis, and C. Lyle. Preventing learning stagnation in ppo by scaling to 1 million parallel environments. *arXiv preprint arXiv:2603.06009*, 2026.
- [41] S. Bharthulwar, S. Tao, and H. Su. Staggered environment resets improve massively parallel on-policy reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025.
- [42] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.
- [43] T. Matiisen, A. Oliver, T. Cohen, and J. Schulman. Teacher–student curriculum learning. *IEEE transactions on neural networks and learning systems*, 31(9):3732–3740, 2019.
- [44] S. Sukhbaatar, Z. Lin, I. Kostrikov, G. Synnaeve, A. Szlam, and R. Fergus. Intrinsic motivation and automatic curricula via asymmetric self-play. *arXiv preprint arXiv:1703.05407*, 2017.
- [45] M. Dennis, N. Jaques, E. Vinitzky, A. Bayen, S. Russell, A. Critch, and S. Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33:13049–13061, 2020.
- [46] J. Parker-Holder, M. Jiang, M. Dennis, M. Samvelyan, J. Foerster, E. Grefenstette, and T. Rocktäschel. Evolving curricula with regret-based environment design. In *International Conference on Machine Learning*, pages 17473–17498. PMLR, 2022.

- [47] A. Rutherford, M. Beukman, T. Willi, B. Lacerda, N. Hawes, and J. Foerster. No regrets: Investigating and improving regret approximations for curriculum discovery. *Advances in Neural Information Processing Systems*, 37:16071–16101, 2024.
- [48] N. Rudin, D. Hoeller, M. Bjelonic, and M. Hutter. Advanced skills by learning locomotion and local navigation end-to-end. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2497–2503. IEEE, 2022.
- [49] Z. Zhuang, Z. Fu, J. Wang, C. G. Atkeson, S. Schwertfeger, C. Finn, and H. Zhao. Robot parkour learning. In *Conference on Robot Learning*, pages 73–92. PMLR, 2023.
- [50] M. Hutter, C. Gehring, A. Lauber, F. Gunther, C. D. Bellicoso, V. Tsounis, P. Fankhauser, R. Diethelm, S. Bachmann, M. Blösch, et al. Anymal-toward legged robots for harsh environments. *Advanced Robotics*, 31(17):918–931, 2017.
- [51] F. Bjelonic, F. Tischhauser, and M. Hutter. Towards bridging the gap: Systematic sim-to-real transfer for diverse legged robots. *arXiv preprint arXiv:2509.06342*, 2025.
- [52] T. J. Boerner, S. Deems, T. R. Furlani, S. L. Knuth, and J. Towns. ACCESS: Advancing innovation: NSF’s advanced cyberinfrastructure coordination ecosystem: Services & support. In *Practice and Experience in Advanced Research Computing (PEARC ’23)*, 2023.

## A Task Specification Details

### A.1 Locomotion

For locomotion, we define  $\rho$  as a discrete categorical distribution  $\rho = (T, L, s_0, g)$  over terrain type  $T$ , terrain level  $L$ , initial state  $s_0$ , and goal state  $g$ . We use 2 seeds per terrain type, 10 difficulty levels, 20 start states, and 20 goal states, yielding 104,000 task configurations.

Terrains from previous works typically have a linear structure where difficulty is determined by parameters such as slope or step height, linearly interpolated from easy to hard. For our procedurally generated terrains, a clear ordering in terms of difficulty may not exist, so the difficulty axis can be interpreted as different seeds.

The terrains used in this paper are shown in the middle and bottom rows of Figure 3, with the exception of Inverted Slope which we omit due to space constraints.

### A.2 Manipulation

We train a separate policy for each manipulation task, whereas locomotion uses a single policy trained jointly across terrains. For manipulation, we define  $\rho$  as a discrete categorical distribution  $\rho = (c, s_0, g)$  over reset configurations  $c$ , the initial state  $s_0$  they emit, and the goal  $g$ . Here  $g$  is fixed across all task configurations to the fully-assembled pose in the fixed-asset frame. The current single-task setup pre-collects a buffer of 32,768 collision-checked configurations balanced across three reset strategies; at training time, every episode reset draws a state from this buffer. We describe each of these reset strategies below:

**Near-Goal.** This reset strategy places the manipulated object, e.g. the nut in `nut-and-bolt` assembly, along its assembly path, e.g. the bolt it threads on. We position the end-effector around a task-specific grasp point using inverse kinematics, randomizing its position, orientation, and gripper aperture. For `rod-in-hole insertion`, the path follows the hole axis and the current fractional range is  $[0, 1.1]$ , where zero is fully seated and values above one include slightly disengaged starts. Other tasks use their own assembly profiles and fractional ranges. For instance, threaded assembly couples axial translation with rotation.

**Stable Grasp.** This reset strategy places the object above the assembly target, at a higher  $z$  position than Near-Goal, and uses the same task-specific grasp setup with a randomized gripper aperture.

**Reaching.** This reset strategy samples the object’s pose anywhere in the workspace, then positions the end-effector to be aligned the object’s grasp frame using inverse kinematics. The gripper aperture is randomized, so the reset does not guarantee that the object is already secured in a stable grasp. This can lead to the object falling anywhere in the workspace, requiring the robot to go fetch the object and then complete the task.

These reset families are used in the ablation in Appendix B.3. Franka `nut-and-bolt` assembly uses the same reset families and reset-buffer construction as UR5e.

**Embodiment-specific simulation and control.** Franka uses joint-position control, whereas UR5e uses operational-space control (OSC). Both use a physics time step of 5 ms (200 Hz), but their policy frequencies differ: Franka runs at 25 Hz with eight physics steps per action, while UR5e runs at 10 Hz with 20 physics steps per action. Thus, Franka requires 2.5 times fewer physics steps per collected policy transition, contributing to its faster training. The reported Franka `nut-and-bolt` assembly runs take approximately 128 hours, compared with 335 hours for UR5e `nut-and-bolt` assembly.

### A.3 Observations, Actions, and Rewards

We describe the observation and action spaces for each embodiment, followed by the reward definitions and weights.

#### A.3.1 Locomotion

**Observations and actions.** The locomotion policy observes base linear and angular velocity, projected gravity, joint positions and velocities, the previous action, the goal command, and a local terrain height scan. Its 12-dimensional action specifies joint-position offsets from the robot’s nominal pose, which are tracked by the joint controllers.

**Rewards.** Since we train a single policy across all terrain types, we use the same reward function across all locomotion tasks. It combines boolean terminal success and failure terms, which contain the bulk of the reward signal, with per-step terms for motion, posture, contacts, and forward alignment:

$$r(s_t, a_t) = \underbrace{\lambda_1 \cdot r_{\text{succ}} + \lambda_2 \cdot r_{\text{fail}}}_{\text{terminal terms}} + \underbrace{\lambda_3 \cdot r_{\text{work}} + \lambda_4 \cdot r_{\text{post}} + \lambda_5 \cdot r_{\text{td}} + \lambda_6 \cdot r_{\text{undc}} + \lambda_7 \cdot r_{\text{fwd}}}_{\text{per-step terms}}. \quad (4)$$

**Term definitions.**

$$\begin{aligned} r_{\text{succ}}(s_t) &= \mathbf{1}[\text{command-success at } t] && (\text{terminal success}) \\ r_{\text{fail}}(s_t) &= \mathbf{1}[\text{drop or base-contact termination at } t] && (\text{terminal failure}) \\ r_{\text{work}}(s_t, a_t) &= \|\tau_t \odot \dot{q}_t\|_1 && (\text{L1 mechanical power}) \\ r_{\text{post}}(s_t) &= \|q_t - q_{\text{nom}}\|_1 && (\text{L1 posture deviation}) \\ r_{\text{td}}(s_t) &= \sum_{f \in \text{feet}} \mathbf{1}[\text{TD}_f(t)] \|v_{f,z}(t)\|_2^2 && (\text{squared touchdown velocity}) \\ r_{\text{undc}}(s_t) &= \sum_{i \in \mathcal{B}_{\text{non-foot}}} \mathbf{1}[\|F_i(t)\|_2 > F_{\min}] && (\text{undesired hard contacts}) \\ r_{\text{fwd}}(s_t) &= \max(0, \langle \hat{v}_t^{(b)}, \hat{g}_t^{(b)} \rangle) && (\text{forward-aligned motion, clamped}) \end{aligned}$$

$r_{\text{fwd}}(s_t)$  is used to promote the robot walking forwards rather than towards the goal but facing backwards. This reward is deactivated within the first 500 steps of training.

**Weights.**

$$\begin{aligned} \lambda_1 (\text{succ}) &= 50, & \lambda_2 (\text{fail}) &= -50, \\ \lambda_3 (\text{work}) &= -10^{-3}, & \lambda_4 (\text{post}) &= -5 \times 10^{-2}, \\ \lambda_5 (\text{td}) &= -2.5 \times 10^{-1}, & \lambda_6 (\text{undc}) &= -5 \times 10^{-2}, \\ \lambda_7 (\text{fwd}) &= 0.3. \end{aligned}$$

#### A.3.2 Manipulation

**Observations and actions.** The teacher observes joint positions, end-effector pose, the previous action, and a scene point cloud of the gripper, held object, and board expressed in the gripper frame. The action comprises three Cartesian translation increments, three axis-angle rotation increments, and one binary gripper open/close command. The arm commands are executed by an operational-space controller. Appendix C.3 describes distillation into an RGB student for deployment.

**Rewards.** The UR5e and Franka assembly tasks share the same reward function, combining assembly success with safety and motion regularization:

$$r(s_t, a_t) = \underbrace{r_{\text{succ}}}_{r_{\text{task}}} + \underbrace{r_{\text{fail}} + r_{\text{act}} + r_{\Delta a} + r_{\dot{q}}}_{r_{\text{reg}}}. \quad (5)$$

**Assembly success and safety regularization.** Let  $S(s_t)$  denote the task-specific assembly-success predicate and  $F(s_t)$  indicate termination from abnormal joint velocities or the held object leaving the configured workspace bounds. The task reward and terminal safety penalty are

$$r_{\text{succ}}(s_t) = \lambda_{\text{succ}} \mathbf{1}[S(s_t)], \quad r_{\text{fail}}(s_t) = -\lambda_{\text{fail}} \mathbf{1}[F(s_t)].$$

The failure penalty is a safety regularizer applied at termination. The abnormal-velocity condition uses twice the configured joint-velocity limits. Success is determined by orientation, lateral-position, and insertion-depth gates using the task’s object geometry; the exact depth bounds and reference points depend on the task.

**Per-step regularizers.** Define  $C(x) = \min(x, 10^4)$  for nonnegative  $x$ . The implementation clamps each squared-norm penalty:

$$\begin{aligned} r_{\text{act}}(s_t, a_t) &= -\lambda_{\text{act}} C(\|a_t\|_2^2) && \text{(action magnitude)} \\ r_{\Delta a}(s_t, a_t) &= -\lambda_{\Delta a} C(\|a_t - a_{t-1}\|_2^2) && \text{(action rate)} \\ r_{\dot{q}}(s_t, a_t) &= -\lambda_{\dot{q}} C(\|\dot{q}_{t,\text{arm}}\|_2^2) && \text{(arm-joint velocity)}. \end{aligned}$$

Here  $\dot{q}_{t,\text{arm}}$  corresponds to the arm joints of the respective robot.

**Configured weights.**

$$\begin{aligned} \lambda_{\text{succ}} &= 100, & \lambda_{\text{fail}} &= 1, & \lambda_{\text{act}} &= 10^{-5}, \\ \lambda_{\Delta a} &= 10^{-4}, & \lambda_{\dot{q}} &= 10^{-4}. \end{aligned}$$

## B Additional Experimental Results

### B.1 Comparison with Sampling for Learnability

Table 1 in the main text compares SFL [47] with SGS on locomotion. We use SFL’s default hyperparameters and report means and 95% confidence intervals over three training seeds. SFL’s final performance is lower than SGS at both scales. At 32K, its best checkpoint comes close, but still underperforms SGS. Notably, performance significantly degrades as training continues, most noticeably at larger scales, whereas SGS monotonically improves during training.

### B.2 SGS Hyperparameters and Configuration Density

**Sampler implementation details.** For each task configuration, we store a window of the latest  $H = 100$  Boolean outcomes in a circular buffer. Our sampling floors the kernel weight before forming logits and enforces a minimum temperature of one:

$$\tilde{w}_i = \max(w_i, \epsilon), \quad \ell_i = \log(\tilde{w}_i + \epsilon), \quad T_{\text{eff}} = \max(T, 1).$$

**Sensitivity experiments.** To evaluate the sensitivity of SGS to each hyperparameter, we vary each SGS hyperparameter one at a time on locomotion at 32K parallel environments, from the nominal setting used during training (in bold) using two training seeds per setting (Table 3). Across the tested hyperparameter settings, the mean success ranges from 0.39 to 0.53, compared with 0.49 for the nominal setting, showing that SGS is not very sensitive to hyperparameters, although it does benefit from a small hyperparameter search. We also compare the Beta kernel against a Normal kernel. Both Normal and Beta kernels achieve similar mean success at 4K environments (0.45 and 0.46, respectively over three training seeds).

We also evaluate how sensitive SGS is to the task configuration density. For task configuration density, each terrain tile uses the product of  $p$  spawn and  $p$  goal patches. Increasing  $p$  from 10 to 20 (i.e. from 52,000 task configurations to 104,000) increases success from 0.38 to 0.49, while  $p = 40$  (i.e. 208,000 task configurations) yields 0.48 at 32K environments. This shows that the sampler benefits from having higher diversity up to a certain extent, after which performance plateaus.

To find the hyperparameter values used during training, we ran a small grid search over different values. For manipulation, a sweep over  $\kappa \in \{1, 5, 10\}$  and  $t \in \{0.33, 0.5, 0.66\}$  with  $T = 2$  at 32K parallel environments yielded  $\kappa = 1$  and  $t = 0.5$  as the best hyperparameters, which performed best by a small margin. Note that we typically do not modify  $T$ , since it is tied to  $\kappa$ , which we do sweep over.

Table 3: **Locomotion sensitivity experiments.** Each entry gives the setting followed by its success rate in parentheses. Bold denotes the nominal values used during training. Hyperparameter experiments use 32K environments and two training seeds per setting. Kernel experiments use 4K environments and three training seeds. Patch-density experiments use 32K environments.

|                  |                 |                                   |                  |           |           |
|------------------|-----------------|-----------------------------------|------------------|-----------|-----------|
| Target $t$       | .33 (.50)       | .50 (.44)                         | <b>.66 (.49)</b> | .80 (.42) | .90 (.52) |
| $\kappa$         | 1 (.44)         | 2 (.47)                           | <b>5 (.49)</b>   | 10 (.46)  |           |
| Temperature $T$  | <b>2 (.49)</b>  | 4 (.53)                           |                  |           |           |
| History $H$      | 25 (.39)        | <b>100 (.49)</b>                  | 400 (.47)        |           |           |
| Floor $\epsilon$ | $10^{-8}$ (.49) | <b><math>10^{-4}</math> (.46)</b> | $10^{-2}$ (.44)  |           |           |
| Kernel           | Normal (.45)    | <b>Beta (.46)</b>                 |                  |           |           |
| Patches $p$      | 10 (.38)        | <b>20 (.49)</b>                   | 40 (.48)         |           |           |

### B.3 Sensitivity to Reset-Distribution Coverage

We evaluate the sensitivity of *SGS* to coverage of resets in terms of entire reset strategies. We retrain UR5e rod-in-hole insertion using buffers constructed from two of the three reset distributions, with three training seeds per setting (Table 4). Removing Near-Goal prevents successful learning in the tested runs. Removing Reaching leaves success on the retained distributions essentially unchanged. Removing Stable Grasp reduces mean success, with one of the three seeds collapsing while the other two retain high performance. These results highlight the importance of near-goal coverage in this reset ablation.

Table 4: rod-in-hole insertion **reset-distribution ablation.** Mean success (%) per reset distribution over three training seeds. Dashes indicate omitted reset distributions.

| Training buffer      | Stable Grasp | Reaching | Near-Goal |
|----------------------|--------------|----------|-----------|
| All three            | 99.5         | 96.0     | 100.0     |
| Without Stable Grasp | —            | 63.8     | 81.3      |
| Without Reaching     | 99.4         | —        | 100.0     |
| Without Near-Goal    | 0.0          | 0.0      | —         |

### B.4 Gravity-Curriculum Ablation

Our main manipulation experiments, including the Franka scaling results, use a gravity curriculum. Training begins in zero gravity, and gravity increases toward its standard value once success exceeds 80% at the current level. After completing the scaling experiments, we trained *SGS* on Franka nut-and-bolt assembly at 1M environments without this curriculum, using one training seed. On the Reaching reset distribution, this run achieves 69.4% success, compared with 70.2% with the curriculum (Table 5), suggesting that the gravity curriculum is unnecessary for strong performance in this setting.

### B.5 Additional UR5e Assembly Tasks for Transfer

We train additional UR5e assembly policies to support real-world transfer. Table 6 reports success from the Reaching initial-state distribution under full gravity ( $1g$ ) across the six tasks in Figure 3. Across samplers, nut-and-bolt assembly is trained with 256K parallel environments and the other five tasks with 64K. These results are separate from the Franka nut-and-bolt assembly and UR5e rod-in-hole insertion scaling experiments.

Table 5: **Gravity-curriculum ablation at 1M parallel environments.** Success on the hardest nut-and-bolt assembly reset strategy. Values are success rates  $\pm$  95% confidence intervals over 512 evaluation episodes, not across training seeds. The no-curriculum ablation uses one training seed.

| Training setting               | Success rate      |
|--------------------------------|-------------------|
| SGS with gravity curriculum    | $0.702 \pm 0.039$ |
| SGS without gravity curriculum | $0.694 \pm 0.040$ |

Table 6: **UR5e evaluation success (%)**. Policies are evaluated from the Reaching initial-state distribution.

| Task type                    | Task                              | Training envs. | Uniform | PLR   | SGS   |
|------------------------------|-----------------------------------|----------------|---------|-------|-------|
| Insertion                    | gear mesh insertion               | 64K            | 94.24   | 95.51 | 96.09 |
|                              | waterproof insertion              | 64K            | 95.31   | 95.90 | 95.80 |
|                              | rectangular-peg-in-hole insertion | 64K            | 92.09   | 84.18 | 92.77 |
|                              | rod-in-hole insertion             | 64K            | 95.90   | 95.41 | 95.70 |
|                              | BNC assembly                      | 64K            | 83.40   | 81.45 | 84.08 |
| Gear alignment and threading | nut-and-bolt assembly             | 256K           | 48.83   | 59.77 | 90.04 |

**Comparison with Franka.** The higher UR5e nut-and-bolt assembly success should be interpreted in the context of its different control setup. UR5e uses operational-space control (OSC), allowing the policy to command Cartesian end-effector motion directly, whereas Franka uses joint-position control and must learn to coordinate its joints to produce the required motion. We attribute much of the performance difference to this control interface. The embodiments and grippers also differ, and policies act at 10 Hz on UR5e versus 25 Hz on Franka.

**Effect of task difficulty.** SGS provides its largest gain on nut-and-bolt assembly: 90.04% success versus 48.83% for uniform sampling and 59.77% for PLR. On the five simpler insertion tasks, SGS performs comparably to uniform sampling. We hypothesize that less forgiving contact dynamics and longer horizons make nut-and-bolt assembly benefit more from concentrating training on configurations with moderate success rates. For simpler insertion tasks, sufficient scale can make uniform sampling effective, although adaptive sampling can improve efficiency: in the separate rod-in-hole insertion scaling experiment, SGS succeeds at 32K environments while uniform sampling and PLR achieve zero success, and all methods reach about 98% at 256K (Figure 5(c)).

## C Sim-to-Real Transfer and Evaluation

We build on the transfer pipeline of OmniReset [12, Appendix A.3]. Below, we describe the hardware changes, simulation settings, and teacher-student design used for our NIST-board tasks, followed by transfer observations and the real-world evaluation protocol.

### C.1 Hardware and Control Setup

We use a UR5e arm with a Robotiq 2F-85 gripper and follow OmniReset’s relative Cartesian operational-space control interface [12, Appendices A.3.5–A.3.6]. Our camera setup uses one third-person camera and one wrist-mounted camera, rather than OmniReset’s three cameras. We describe the point-cloud teacher and RGB student in Appendix C.3.

**Board surface.** The NIST board is made of polished acrylic. When the wrist camera faces the board directly, the robot observes its own reflection, which is out of distribution. The low surface friction also impairs policies that use the board to reorient the object. For these reasons, we covered the board with tape during evaluation.

## C.2 Simulation Modeling and Domain Randomization

**Domain randomization.** We follow OmniReset’s domain-randomization setup [12], adding small Gaussian noise to the point-cloud observations and randomizing object masses around their measured real-world values.

**Gripper pad friction.** We use a fixed pad friction of  $\mu = 2$  with the max friction-combine mode to reduce unrealistic grasps while retaining stable training.

OmniReset [12] gave the Robotiq 2F-85 pads a friction coefficient of  $\mu = 100$  with the max combine mode. This makes grasping forgiving and training easy, but we found it to be a significant source of sim-to-real gap. Any grasp that applies normal force succeeds in simulation, because the object stays frozen between the pads even when the grasp is physically implausible (Figure 9).



Figure 9: Example of an unrealistic grasp produced by  $100\mu$  gripper pads

Lowering the friction removes these grasps, but in our initial experiments policies trained from scratch at low pad friction failed to learn at all. To find the lowest friction that still trains, we trained rod-in-hole insertion from scratch at fixed pad frictions of  $\mu = 0.5, 1, 1.5$ , and  $2$ , keeping the max combine mode and all other settings identical (Figure 10). Only  $\mu = 2$  trained stably. We therefore use  $\mu = 2$  for all training runs.(Figure 9).

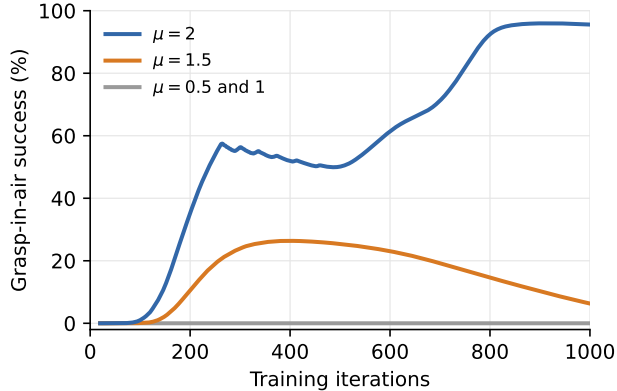


Figure 10: **Training rod-in-hole insertion from scratch at pad friction  $\mu = 0.5, 1, 1.5$ , and  $2$ .** Grasp-in-air success rate over training (max combine, pad friction fixed for the whole run)

## C.3 Teacher-Student Distillation

For UR5e transfer, *SGS* trains a point-cloud teacher in simulation and distills it into an RGB student for deployment. The teacher actor observes arm joint positions, the previous action, and a scene point cloud expressed in the gripper frame; the critic additionally receives privileged state, including

joint velocities and the sampled domain-randomization parameters. We use a point-cloud teacher rather than one that observes object poses because geometric observations narrow the gap between what the teacher and the RGB-based student can perceive, which makes the teacher’s behavior easier to imitate from images. The student observes side and wrist RGB images, each encoded by a separate ResNet-18, together with arm joint positions, end-effector pose, and the previous action. OmniReset collects a dataset and trains the student with behavioral cloning [12]. We instead use DAgger for faster training and a simpler workflow, eliminating the separate dataset-collection step. During distillation, we use OmniReset’s visual randomization and image augmentation [12, Table 2]. Only the RGB student is run on hardware, without real-world data or fine-tuning.

**Online DAgger and held-out evaluation.** Each iteration collects one step across all environments, updates the student toward the teacher’s actions, and discards the data. This limits GPU memory use, a bottleneck when simulation and rendering run alongside training. However, training success can be misleading: updates from the immediately preceding step can guide the student’s next action toward the teacher, helping it solve a task it cannot yet perform without these updates. We therefore reserve 6.67% of environments for evaluation and exclude their data from gradient updates. Success in these held-out environments lags training success, revealing the need for longer distillation (Figure 11).

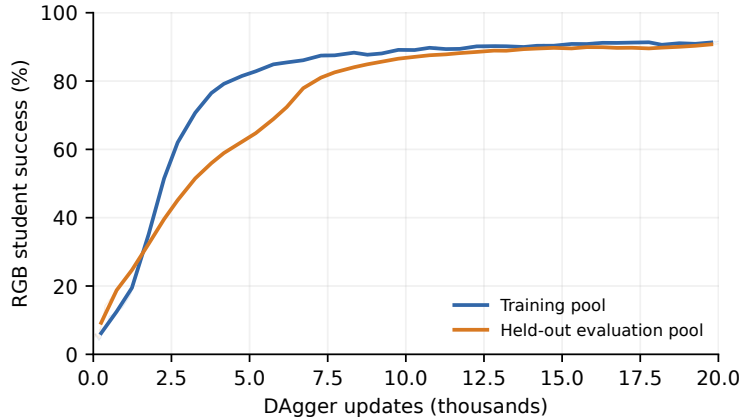


Figure 11: **Training and held-out evaluation during RGB distillation on nut-and-bolt assembly.** Evaluation success lags training success early on and catches up with continued training; curves show 500-update averages from one run.

**Symmetry-aware resets and success criteria.** We randomize resets over each object’s equivalent roll, pitch, and yaw angles. Discrete angles are sampled uniformly; continuously symmetric angles are sampled uniformly over  $[0, 360^\circ)$ . The success reward accepts any of the same equivalent goal orientations. These rotations are defined in the object-local frame relative to the nominal configuration.

Using the same symmetries for resets and rewards avoids favoring an orientation that the RGB-based student cannot distinguish from an equivalent one. Point-cloud observations alone do not remove this ambiguity if resets or rewards still impose a preferred orientation. Such preferences can also induce unnecessary reorientation behaviors that transfer poorly, as illustrated by the nut-flipping failure in Appendix C.4.

On OmniReset peg insertion, the distilled RGB student achieves approximately 55% success [12, Table 1]. Using a point-cloud teacher with symmetric resets and rewards increases student success to 98–99%. Each grasp reset uniformly samples one of eight equivalent object orientations, and the reward accepts successful insertion in any of them. This reduces the teacher–student observability gap by removing orientation distinctions unavailable to the RGB student, making the teacher easier to imitate.

Table 7: **RGB-student success in simulation on OmniReset peg insertion.** The pose-based baseline is reported in OmniReset [12, Table 1]; the symmetry-aware point-cloud teacher yields 98% success after distillation.

| Teacher configuration          | RGB student success (%) |
|--------------------------------|-------------------------|
| Pose-based (OmniReset)         | 55.45                   |
| Eight-orientation, point-cloud | 98                      |

#### C.4 Transfer Observations and Policy Behaviors

A consistent lesson across tasks was that policies which commit to a clean grasp and insert directly transfer better than ones that exploit simulator contact dynamics. For `rod-in-hole insertion`, adding 1 mm per-step Gaussian noise to the point-cloud observation further helped by encouraging the policy to re-aim after a miss and wiggle the rod when it was not fully seated. For `nut-and-bolt assembly`, the policy repeatedly dropped the nut onto the bolt instead of seating it, flipping the nut each time. We speculate this was due to the teacher reward, which only counted one upright nut orientation for success despite the nut being symmetric. Removing this constraint and using appropriately tuned action scaling on the 6-DOF end-effector control space produced a policy that completed the task.

#### C.5 Real-World Evaluation Protocol

Figure 4 shows hardware trajectories, and Table 2 reports task-level results. We follow the evaluation protocol and metric definitions of OmniReset [12, Appendix A.4], reporting overall success rate, first-try success rate, and throughput, with the differences noted below.

**Initialization.** We sample initial object configurations by uniformly randomizing object positions over the NIST board. The board is fixed to the table with command strips. In simulation the receptive objects are static, and the policy learns to reorient the insertive object relative to a fixed reference. If the board is not secured, the receptive objects can shift during contact, which causes failures.

**Trials.** We run the policy continuously for approximately 83 minutes on the nut, 42 minutes on the rod, and 15 minutes on the gear, collecting 48, 49, and 50 trajectories, respectively.

**Metrics.** A trial is a first-try success if the policy completes the insertion on its first attempt. A failed insertion followed by another attempt (e.g., re-poking) counts as a retry. Unlike OmniReset, which counts any re-grasp as a retry, we do not penalize re-grasps that occur before the first insertion attempt. Throughput is the number of successful trajectories divided by the total evaluation time, which includes reset time.